

NETWORK LAB — ASSIGNMENT 2

Computer Networks (PCCST501)

Name: Prarthana Prakas

Rollno: 53

Class: S5 CSE A

Experiment 1: Hello — Client/Server Communication (TCP Sockets)

To implement a basic client–server application using TCP sockets in C, in which the client connects to the server, sends a greeting message (“Hello Server”), and receives a reply (“Hello Client”) from the server, thereby demonstrating a reliable two-way communication channel between two processes over a network.

Software Used

```
gcc server.c -o server
```

```
gcc client.c -o client
```

```
./server (run in Terminal 1)
```

```
./client (run in Terminal 2)
```

Algorithm

Server:

- 1 Create a TCP socket using `socket()`.
- 2 Bind the socket to IP address `INADDR_ANY` and port 8080 using `bind()`.
- 3 Put the socket into listening mode using `listen()`.
- 4 Accept an incoming client connection using `accept()`.
- 5 Receive the message sent by the client using `recv()`.
- 6 Display the received message and send the reply “Hello Client” using `send()`.
- 7 Close the client and server sockets.

Client:

- 1 Create a TCP socket using `socket()`.
- 2 Specify the server's IP address (127.0.0.1) and port number (8080).
- 3 Connect to the server using `connect()`.
- 4 Send the message “Hello Server” using `send()`.
- 5 Receive the server's reply using `recv()` and display it.
- 6 Close the socket.

Server Code (server.c)

```
#include <string.h>

#include <winsock2.h>

#define PORT 8080

int main()
{
    WSADATA wsa;

    SOCKET serverSocket, clientSocket;

    struct sockaddr_in serverAddr, clientAddr;

    int addrSize;

    char buffer[1024];

    WSStartup(MAKEWORD(2, 2), &wsa);

    serverSocket = socket(AF_INET, SOCK_STREAM, 0);

    serverAddr.sin_family = AF_INET;

    serverAddr.sin_addr.s_addr = INADDR_ANY;

    serverAddr.sin_port = htons(PORT);

    bind(serverSocket, (struct sockaddr *)&serverAddr,
        sizeof(serverAddr));

    listen(serverSocket, 5);

    printf("Server is waiting for connection...\n");

    addrSize = sizeof(clientAddr);

    clientSocket = accept(serverSocket,
        (struct sockaddr *)&clientAddr,
        &addrSize);

    recv(clientSocket, buffer, sizeof(buffer), 0);

    printf("Client says: %s\n", buffer);

    strcpy(buffer, "Hello Client");

    send(clientSocket, buffer, strlen(buffer) + 1, 0);

    closesocket(clientSocket);

    closesocket(serverSocket);

    WSACleanup();

    return 0;
}
```

Client Code (client.c)

```
#include <stdio.h>

#include <string.h>
```

```
#include <winsock2.h>

#define PORT 8080

int main()
{
    WSADATA wsa;

    SOCKET clientSocket;

    struct sockaddr_in serverAddr;

    char buffer[1024];

    WSStartup(MAKEWORD(2, 2), &wsa);

    clientSocket = socket(AF_INET, SOCK_STREAM, 0);

    serverAddr.sin_family = AF_INET;

    serverAddr.sin_port = htons(PORT);

    serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

    connect(clientSocket,

            (struct sockaddr *)&serverAddr,

            sizeof(serverAddr));

    strcpy(buffer, "Hello Server");

    send(clientSocket, buffer, strlen(buffer) + 1, 0);

    recv(clientSocket, buffer, sizeof(buffer), 0);

    printf("Server says: %s\n", buffer);

    closesocket(clientSocket);

    WSACleanup();

    return 0;
}
```

Output

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\prart\OneDrive\Documents\assignment3>client1.exe
Server says: Hello Client

C:\Users\prart\OneDrive\Documents\assignment3>
```

```
C:\Users\prart\OneDrive\Doc >
Server is waiting for connection...
Client says: Hello Server

Process returned 0 (0x0)   execution time : 14.149 s
Press any key to continue.
```

Conclusion

The client–server program was implemented successfully using TCP sockets. The server was started first and began listening on port 8080, after which the client connected to it, sent the message “Hello Server”, and received the reply “Hello Client”. This confirms that a reliable, connection-oriented, two-way communication channel was successfully established between the client and the server processes

Experiment 2:

Arithmetic Server

Aim

To implement a client–server application in which the client sends two integers to the server, and the server computes and returns their sum, difference, product, and quotient, demonstrating exchange of numeric data and remote computation over a TCP socket connection.

Software Used

GCC Compiler, Linux Terminal (WSL / Ubuntu)

Commands Used

```
gcc server.c -o server
```

```
gcc client.c -o client
```

```
./server (run in Terminal 1)
```

```
./client (run in Terminal 2)
```

Algorithm Server:

- 1 Create a TCP socket and enable SO_REUSEADDR using setsockopt().
- 2 Bind the socket to INADDR_ANY and port 8081 using bind().
- 3 Put the socket into listening mode using listen().
- 4 Accept an incoming client connection using accept().
- 5 Receive the two integers (sent as a space-separated string) using recv().
- 6 Parse the integers using sscanf() and compute sum, difference, product, and quotient (checking for division by zero).
- 7 Format the results into a string and send them back to the client using send().
- 8 Close the client and server sockets.

Client:

- 1 Create a TCP socket using socket().
- 2 Specify the server's IP address (127.0.0.1) and port number (8081).
- 3 Connect to the server using connect().
- 4 Read two integers from the user and format them into a message string.
- 5 Send the message to the server using send().

6 Receive the computed results from the server using `recv()` and display them.

7 Close the socket.

Server Code (server.c)

```
#include <stdio.h>

#include <winsock2.h>

#define PORT 8080

int main()
{
    WSADATA wsa;

    SOCKET serverSocket, clientSocket;

    struct sockaddr_in serverAddr, clientAddr;

    int addrSize;


    int a, b;

    int sum, difference, product;

    float division;


    /* Initialize Winsock */
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0)
    {
        printf("WSAStartup failed.\n");

        return 1;
    }


    /* Create socket */

    serverSocket = socket(AF_INET, SOCK_STREAM, 0);


    if (serverSocket == INVALID_SOCKET)
    {
        printf("Socket creation failed.\n");

        WSACleanup();

        return 1;
    }


    /* Configure server address */

    serverAddr.sin_family = AF_INET;

    serverAddr.sin_addr.s_addr = INADDR_ANY;

    serverAddr.sin_port = htons(PORT);
```

```

/* Bind socket */

if (bind(serverSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr)) == SOCKET_ERROR)
{
    printf("Bind failed. Error code: %d\n", WSAGetLastError());
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}

/* Listen for connections */

if (listen(serverSocket, 5) == SOCKET_ERROR)
{
    printf("Listen failed.\n");
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}

printf("=====\n");
printf("    CALCULATOR SERVER\n");
printf("=====\n");
printf("Server is waiting for connection...\n");

/* Accept client connection */

addrSize = sizeof(clientAddr);

clientSocket = accept(serverSocket,
                    (struct sockaddr *)&clientAddr,
                    &addrSize);

if (clientSocket == INVALID_SOCKET)
{
    printf("Accept failed.\n");

    closesocket(serverSocket);
    WSACleanup();
    return 1;
}

```

```

printf("Client connected successfully.\n");

/* Receive first integer */
if (recv(clientSocket, (char *)&a, sizeof(a), 0) <= 0)
{
    printf("Failed to receive first number.\n");
    closesocket(clientSocket);
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}

/* Receive second integer */
if (recv(clientSocket, (char *)&b, sizeof(b), 0) <= 0)
{
    printf("Failed to receive second number.\n");
    closesocket(clientSocket);
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}

printf("Received numbers: %d and %d\n", a, b);

/* Perform calculations */
sum = a + b;
difference = a - b;
product = a * b;
if (b != 0)
{
    division = (float)a / b;
}
else
{
    division = 0;
}

/* Display results on server */
printf("\nResults:\n");
printf("Sum      = %d\n", sum);
printf("Difference = %d\n", difference);
printf("Product   = %d\n", product);

```

```

if (b != 0)
{
    printf("Division  = %.2f\n", division);
}
else
{
    printf("Division  = Cannot divide by zero\n");
}

/* Send results to client */

send(clientSocket, (char *)&sum, sizeof(sum), 0);
send(clientSocket, (char *)&difference, sizeof(difference), 0);
send(clientSocket, (char *)&product, sizeof(product), 0);
send(clientSocket, (char *)&division, sizeof(division), 0);

printf("\nResults sent to client successfully.\n");

/* Close sockets */

closesocket(clientSocket);
closesocket(serverSocket);

/* Clean up Winsock */

WSACleanup();

printf("\nServer closed.\n");

return 0;
}

```

Client code(client.c)

```

#include <stdio.h>

#include <winsock2.h>

#define PORT 8080

int main()
{
    WSADATA wsa;

    SOCKET clientSocket;

    struct sockaddr_in serverAddr;

    int a, b;

    int sum, difference, product;

    float division;

    WSStartup(MAKEWORD(2, 2), &wsa);

    clientSocket = socket(AF_INET, SOCK_STREAM, 0);

```

```

// Server address

serverAddr.sin_family = AF_INET;

serverAddr.sin_port = htons(PORT);

serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

// Connect to server

connect(clientSocket,

        (struct sockaddr *)&serverAddr,

        sizeof(serverAddr));

// Input

printf("Enter two integers: ");

scanf("%d %d", &a, &b);

// Send integers

send(clientSocket, (char *)&a, sizeof(a), 0);

send(clientSocket, (char *)&b, sizeof(b), 0);

// Receive results

recv(clientSocket, (char *)&sum, sizeof(sum), 0);

recv(clientSocket, (char *)&difference, sizeof(difference), 0);

recv(clientSocket, (char *)&product, sizeof(product), 0);

recv(clientSocket, (char *)&division, sizeof(division), 0);

// Display results

printf("\nAddition    = %d\n", sum);

printf("Subtraction  = %d\n", difference);

printf("Multiplication = %d\n", product);

if (b != 0)

    printf("Division    = %.2f\n", division);

else

    printf("Division    = Cannot divide by zero\n");

closesocket(clientSocket);

WSACleanup();

return 0;

}

```

Output

```
C:\Windows\System32\Fsmc >
Microsoft Windows [Version 10.0.26280.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\pratt\OneDrive\Documents\assignment3>client2.exe
Enter two integers: 4
2 4 3 5

Addition      = 6
Subtraction   = 2
Multiplication = 8
Division      = 2.00

C:\Users\pratt\OneDrive\Documents\assignment3>
```

```
C:\Users\pratt\OneDrive\Documents\assignment3>
=====
          CALCULATOR SERVER
=====
Server is waiting for connection...
Client connected successfully.
Received numbers: 4 and 2

Results:
Sum      = 6
Difference = 2
Product  = 8
Division = 2.00

Results sent to client successfully.
Server closed.

Process returned 0 (0x0)   execution time : 0.044 s
Press any key to continue.
```

Conclusion

The arithmetic server was successfully implemented and tested.

Experiment 3: String Processing Server

Aim

To implement a client–server application in which the client sends a string to the server, and the server computes the length of the string, converts it to uppercase, and reverses it, then returns all the results to the client, demonstrating string processing over a TCP socket connection.

Software Used

GCC Compiler, Linux Terminal (WSL / Ubuntu)

Commands Used

- `gcc server.c -o server`
- `gcc client.c -o client`
- `./server` (run in Terminal 1)
- `./client` (run in Terminal 2)

Algorithm

Server:

1. Create a TCP socket and enable `SO_REUSEADDR` using `setsockopt()`.
2. Bind the socket to `INADDR_ANY` and port 8082 using `bind()`.
3. Put the socket into listening mode using `listen()`.
4. Accept an incoming client connection using `accept()`.
5. Receive the string sent by the client using `recv()`.
6. Compute the length of the string, convert it to uppercase, and reverse it.
7. Format all three results into a string and send them back to the client using `send()`.
8. Close the client and server sockets.

Client:

1. Create a TCP socket using `socket()`.
2. Specify the server's IP address (127.0.0.1) and port number (8082).
3. Connect to the server using `connect()`.
4. Read a string from the user using `fgets()` and strip the trailing newline.
5. Send the string to the server using `send()`.
6. Receive the processed results from the server using `recv()` and display them.
7. Close the socket.

Server Code (server.c)

```
#include <stdio.h>

#include <string.h>

#include <ctype.h>

#include <winsock2.h>

#define PORT 8080

int main()
{
    WSADATA wsa;

    SOCKET serverSocket, clientSocket;

    struct sockaddr_in serverAddr, clientAddr;

    int addrSize;

    char str[1024];

    char uppercase[1024];

    char reverse[1024];

    int length;

    int i;

    WSStartup(MAKEWORD(2, 2), &wsa);

    serverSocket = socket(AF_INET, SOCK_STREAM, 0);

    serverAddr.sin_family = AF_INET;

    serverAddr.sin_addr.s_addr = INADDR_ANY;

    serverAddr.sin_port = htons(PORT);

    bind(serverSocket, (struct sockaddr *)&serverAddr,
        sizeof(serverAddr));

    listen(serverSocket, 5);

    printf("Server is waiting for connection...\n");
```

```

addrSize = sizeof(clientAddr);

clientSocket = accept(serverSocket,
                      (struct sockaddr *)&clientAddr,
                      &addrSize);

recv(clientSocket, str, sizeof(str), 0);

length = strlen(str);

for (i = 0; i < length; i++)
{
    uppercase[i] = toupper((unsigned char)str[i]);
}

uppercase[length] = '\0';

for (i = 0; i < length; i++)
{
    reverse[i] = str[length - 1 - i];
}

reverse[length] = '\0';

send(clientSocket, (char *)&length, sizeof(length), 0);

send(clientSocket, uppercase, sizeof(uppercase), 0);

send(clientSocket, reverse, sizeof(reverse), 0);

closesocket(clientSocket);

closesocket(serverSocket);

WSACleanup();

return 0;
}

```

Client code(client.c)

```

#include <stdio.h>

#include <string.h>

#include <winsock2.h>

#define PORT 8080

int main()
{
    WSADATA wsa;

    SOCKET clientSocket;

    struct sockaddr_in serverAddr;

    char str[1024];

    char uppercase[1024];

    char reverse[1024];

```

```

int length;

// Initialize Winsock
WSAStartup(MAKEWORD(2, 2), &wsa);

// Create socket
clientSocket = socket(AF_INET, SOCK_STREAM, 0);

// Server address
serverAddr.sin_family = AF_INET;

serverAddr.sin_port = htons(PORT);

serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

// Connect to server
connect(clientSocket,

        (struct sockaddr *)&serverAddr,

        sizeof(serverAddr));

// Input string
printf("Enter a string: ");

fgets(str, sizeof(str), stdin);

// Remove newline
str[strcspn(str, "\n")] = '\0';

// Send string
send(clientSocket, str, sizeof(str), 0);

// Receive results
recv(clientSocket, (char *)&length, sizeof(length), 0);

recv(clientSocket, uppercase, sizeof(uppercase), 0);

recv(clientSocket, reverse, sizeof(reverse), 0);

// Display results
printf("\nLength   = %d\n", length);

printf("Uppercase = %s\n", uppercase);

printf("Reverse  = %s\n", reverse);

closesocket(clientSocket);

WSACleanup();

return 0;
}

```

output

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26208.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\prart\OneDrive\Documents\assignment3>client3.exe
Enter a string: Hello

Length      = 5
Uppercase   = HELLO
Reverse     = olleH

C:\Users\prart\OneDrive\Documents\assignment3>
```

```
C:\Users\prart\OneDrive\Documents>
Server is waiting for connection...
Process returned 0 (0x0)   execution time : 22.764 s
Press any key to continue.
```

Conclusion

The string processing server was successfully implemented and tested

Experiment 4: Array Processing Server

Aim

To implement a client–server application in which the client sends an array of N integers to the server, and the server computes the sum, average, maximum, and minimum of the array elements, demonstrating array data transfer and aggregate computation over a TCP socket connection.

Software Used

GCC Compiler, Linux Terminal (WSL / Ubuntu)

Commands Used

- gcc server.c -o server
- gcc client.c -o client
- ./server (run in Terminal 1)
- ./client (run in Terminal 2)

Algorithm

Server:

1. Create a TCP socket and enable SO_REUSEADDR using setsockopt().
2. Bind the socket to INADDR_ANY and port 8083 using bind().
3. Put the socket into listening mode using listen().
4. Accept an incoming client connection using accept().
5. Receive the message containing N followed by N integers (space-separated) using recv().
6. Tokenize the message using strtok() to extract N and the array elements.
7. Compute the sum, average, maximum, and minimum of the array elements.
8. Format the results into a string and send them back to the client using send().
9. Close the client and server sockets.

Client:

1. Create a TCP socket using socket().
2. Specify the server's IP address (127.0.0.1) and port number (8083).

3. Connect to the server using `connect()`.
4. Read the number of elements `N` and then `N` integers from the user.
5. Build a single message string containing `N` followed by the array elements.
6. Send the message to the server using `send()`.
7. Receive the computed results from the server using `recv()` and display them.
8. Close the socket.

Server Code (server.c)

```
#include <stdio.h>

#include <stdlib.h>

#include <winsock2.h>

#pragma comment(lib, "ws2_32.lib")

#define PORT 8080

int recvAll(SOCKET sock, char *buffer, int size)
{
    int total = 0;
    int bytes;

    while (total < size)
    {
        bytes = recv(sock, buffer + total, size - total, 0);

        if (bytes <= 0)
            return 0;

        total += bytes;
    }

    return 1;
}

int main()
{
```

```

WSADATA wsa;

SOCKET serverSocket, clientSocket;

struct sockaddr_in server, client;

int clientSize;

int n, i;

int arr[100];


int sum, maximum, minimum;

double average;


/* Start Winsock */
WSAStartup(MAKEWORD(2, 2), &wsa);


/* Create socket */
serverSocket = socket(AF_INET, SOCK_STREAM, 0);


if (serverSocket == INVALID_SOCKET)
{
    printf("Socket creation failed.\n");
    return 1;
}


/* Server address */
server.sin_family = AF_INET;
server.sin_addr.s_addr = INADDR_ANY;
server.sin_port = htons(PORT);


/* Bind */
if (bind(serverSocket, (struct sockaddr *)&server,
        sizeof(server)) == SOCKET_ERROR)
{
    printf("Bind failed.\n");
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}


/* Listen */

```

```
listen(serverSocket, 1);

printf("Server waiting for client...\n");

/* Accept client */
clientSize = sizeof(client);
clientSocket = accept(serverSocket,
                      (struct sockaddr *)&client,
                      &clientSize);

if (clientSocket == INVALID_SOCKET)
{
    printf("Accept failed.\n");
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}

printf("Client connected.\n");

/* Receive N */
if (!recvAll(clientSocket, (char *)&n, sizeof(int)))
{
    printf("Error receiving N.\n");
    closesocket(clientSocket);
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}

/* Receive array */
if (!recvAll(clientSocket, (char *)arr, n * sizeof(int)))
{
    printf("Error receiving array.\n");
    closesocket(clientSocket);
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}
```

```

}

/* Calculate results */

sum = 0;
maximum = arr[0];
minimum = arr[0];

for (i = 0; i < n; i++)
{
    sum += arr[i];

    if (arr[i] > maximum)
        maximum = arr[i];

    if (arr[i] < minimum)
        minimum = arr[i];
}

average = (double)sum / n;

/* Send results */

send(clientSocket, (char *)&sum, sizeof(int), 0);

send(clientSocket, (char *)&average, sizeof(double), 0);

send(clientSocket, (char *)&maximum, sizeof(int), 0);

send(clientSocket, (char *)&minimum, sizeof(int), 0);

printf("Results sent to client.\n");

/* Close sockets */

closesocket(clientSocket);

closesocket(serverSocket);

WSACleanup();

return 0;
}

```

Client code(client.c)

```

#pragma comment(lib, "ws2_32.lib")

#define PORT 8080

int recvAll(SOCKET sock, char *buffer, int size)
{

```

```

int total = 0;

int bytes;

while (total < size)
{
    bytes = recv(sock, buffer + total, size - total, 0);

    if (bytes <= 0)
        return 0;

    total += bytes;
}

return 1;
}

int main()
{
    WSADATA wsa;
    SOCKET clientSocket;
    struct sockaddr_in server;

    int n, i;
    int arr[100];

    int sum, maximum, minimum;
    double average;

    /* Start Winsock */
    WSASStartup(MAKEWORD(2, 2), &wsa);

    /* Create socket */
    clientSocket = socket(AF_INET, SOCK_STREAM, 0);

    if (clientSocket == INVALID_SOCKET)
    {
        printf("Socket creation failed.\n");
        return 1;
    }

```

```

}

/* Server address */
server.sin_family = AF_INET;
server.sin_addr.s_addr = inet_addr("127.0.0.1");
server.sin_port = htons(PORT);

/* Connect to server */
if (connect(clientSocket,
            (struct sockaddr *)&server,
            sizeof(server)) == SOCKET_ERROR)
{
    printf("Connection failed.\n");
    closesocket(clientSocket);
    WSACleanup();
    return 1;
}

printf("Connected to server.\n");

/* Input N */
printf("Enter number of integers: ");
scanf("%d", &n);

if (n <= 0 || n > 100)
{
    printf("Enter N between 1 and 100.\n");
    closesocket(clientSocket);
    WSACleanup();
    return 1;
}

/* Input array */
printf("Enter %d integers:\n", n);

for (i = 0; i < n; i++)
{
    scanf("%d", &arr[i]);

```

```

}

/* Send N */
send(clientSocket, (char *)&n, sizeof(int), 0);

/* Send array */
send(clientSocket, (char *)arr, n * sizeof(int), 0);

/* Receive results */
if (!recvAll(clientSocket, (char *)&sum, sizeof(int)) ||
    !recvAll(clientSocket, (char *)&average, sizeof(double)) ||
    !recvAll(clientSocket, (char *)&maximum, sizeof(int)) ||
    !recvAll(clientSocket, (char *)&minimum, sizeof(int)))
{
    printf("Error receiving results.\n");
    closesocket(clientSocket);
    WSACleanup();
    return 1;
}

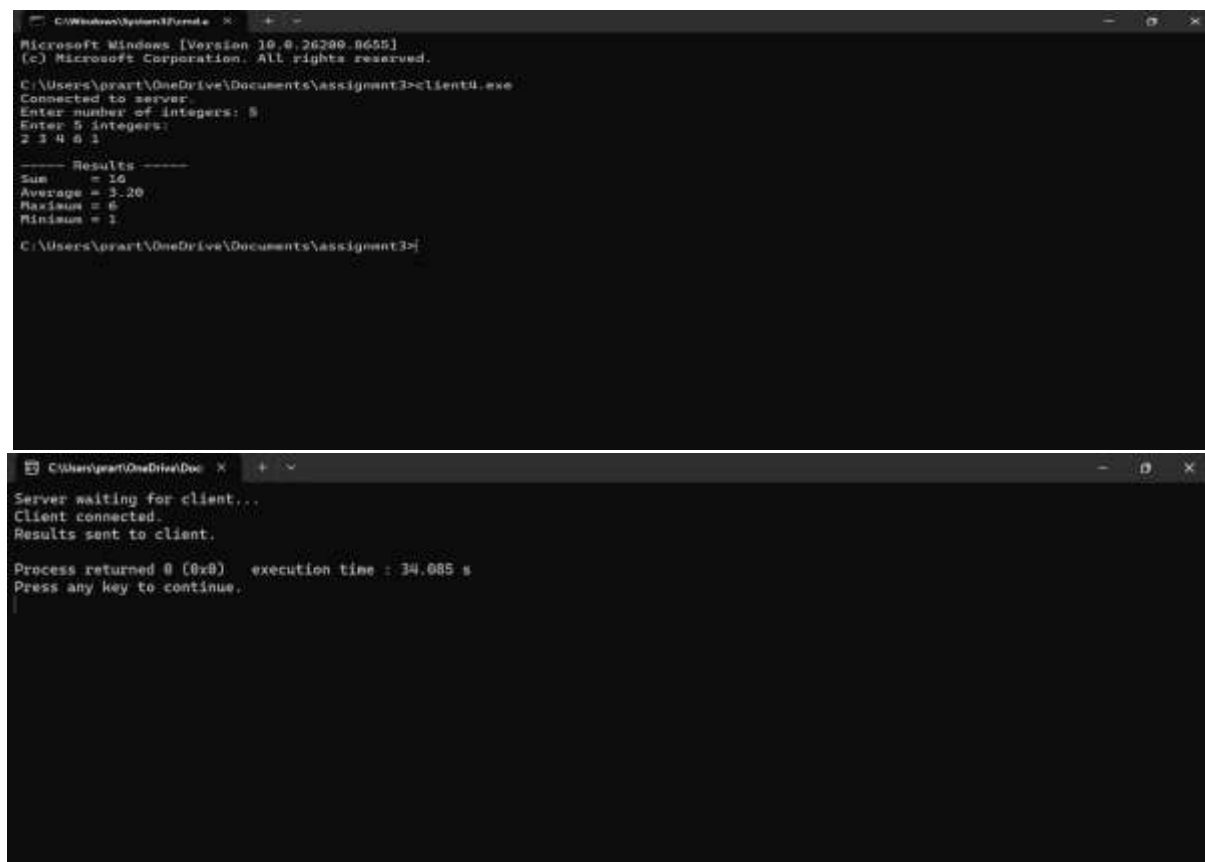
/* Display results */
printf("\n----- Results ----- \n");
printf("Sum    = %d\n", sum);
printf("Average = %.2f\n", average);
printf("Maximum = %d\n", maximum);
printf("Minimum = %d\n", minimum);

/* Close socket */
closesocket(clientSocket);
WSACleanup();

return 0;
}

```

output



```
C:\Windows\System32\Fsmc >
Microsoft Windows [Version 10.0.20290.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\prart\OneDrive\Documents\assignment3>clientd.exe
Connected to server.
Enter number of integers: 5
Enter 5 integers:
2 3 4 5 1

----- Results -----
Sum      = 15
Average  = 3.00
Maximum  = 5
Minimum  = 1

C:\Users\prart\OneDrive\Documents\assignment3>
```

```
C:\Users\prart\OneDrive\Doc >
Server waiting for client...
Client connected.
Results sent to client.

Process returned 0 (0x0)   execution time : 34.085 s
Press any key to continue.
```

conclusion

The array processing server was successfully implement

Experiment 5: Matrix Analyzer

Aim

To implement a client-server application in which the client sends a square matrix to the server, and the server determines the type of matrix (diagonal, upper triangular, lower triangular, or general), and computes the sum of its elements and its trace, demonstrating two-dimensional array processing over a TCP socket connection.

Software Used

GCC Compiler, Linux Terminal (WSL / Ubuntu)

Commands Used

- `gcc server.c -o server`
- `gcc client.c -o client`
- `./server` (run in Terminal 1)
- `./client` (run in Terminal 2)

Algorithm

Server:

1. Create a TCP socket and enable `SO_REUSEADDR` using `setsockopt()`.
2. Bind the socket to `INADDR_ANY` and port 8084 using `bind()`.
3. Put the socket into listening mode using `listen()`.
4. Accept an incoming client connection using `accept()`.
5. Receive the message containing `N` followed by the $N \times N$ matrix elements (space-separated) using `recv()`.
6. Tokenize the message using `strtok()` to reconstruct `N` and the matrix.
7. Check the matrix for diagonal, upper triangular, and lower triangular properties, and classify it accordingly.
8. Compute the sum of all elements and the trace (sum of diagonal elements) of the matrix.
9. Format the matrix type, sum, and trace into a string and send them back to the client using `send()`.

10. Close the client and server sockets.

Client:

1. Create a TCP socket using `socket()`.
2. Specify the server's IP address (127.0.0.1) and port number (8084).
3. Connect to the server using `connect()`.
4. Read the order N and then the N×N matrix elements from the user.
5. Build a single message string containing N followed by all the matrix elements.
6. Send the message to the server using `send()`.
7. Receive the computed results (matrix type, sum, trace) from the server using `recv()` and display them.
8. Close the socket.

Server Code (server.c)

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <winsock2.h>


#define PORT 8080

#define MAX 10


int main()
{
    WSADATA wsa;

    SOCKET serverSocket, clientSocket;

    struct sockaddr_in serverAddr;


    int n, matrix[MAX][MAX];

    int i, j;

    int sum = 0, trace = 0;

    int upper = 1, lower = 1, diagonal = 1;

    char result[1024];
```

```
WSAStartup(MAKEWORD(2, 2), &wsa);
```

```
serverSocket = socket(AF_INET, SOCK_STREAM, 0);
```

```
if (serverSocket == INVALID_SOCKET)
{
    printf("Socket Creation Failed\n");
    return 1;
}
```

```
printf("Socket Created Successfully...\n");
```

```
serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(PORT);
```

```
if (bind(serverSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr)) == SOCKET_ERROR)
{
    printf("Bind Failed\n");
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}
```

```
printf("Bind Successful...\n");
```

```
listen(serverSocket, 5);
```

```
printf("Waiting for Client...\n");
```

```
clientSocket = accept(serverSocket, NULL, NULL);
```

```
if (clientSocket == INVALID_SOCKET)
{
    printf("Accept Failed\n");
    return 1;
}
```

```
}
```

```
printf("Client Connected...\n");
```

```
recv(clientSocket, (char *)&n, sizeof(n), 0);
```

```
recv(clientSocket, (char *)matrix, sizeof(matrix), 0);
```

```
printf("\nReceived Matrix:\n");
```

```
for (i = 0; i < n; i++)
```

```
{
```

```
    for (j = 0; j < n; j++)
```

```
    {
```

```
        printf("%d ", matrix[i][j]);
```

```
        sum += matrix[i][j];
```

```
        if (i == j)
```

```
            trace += matrix[i][j];
```

```
        if (i != j && matrix[i][j] != 0)
```

```
            diagonal = 0;
```

```
        if (i > j && matrix[i][j] != 0)
```

```
            upper = 0;
```

```
        if (i < j && matrix[i][j] != 0)
```

```
            lower = 0;
```

```
    }
```

```
    printf("\n");
```

```
}
```

```
if (diagonal)
```

```
    strcpy(result, "Matrix Type : Diagonal Matrix\n");
```

```
else if (upper)
```

```
    strcpy(result, "Matrix Type : Upper Triangular Matrix\n");
```

```
else if (lower)
```

```
    strcpy(result, "Matrix Type : Lower Triangular Matrix\n");
```

```

else

    strcpy(result, "Matrix Type : General Matrix\n");

char temp[200];

sprintf(temp, "Sum of Elements : %d\nTrace : %d\n",
        sum, trace);

strcat(result, temp);

printf("\n%s", result);

send(clientSocket, result, strlen(result) + 1, 0);

closesocket(clientSocket);
closesocket(serverSocket);
WSACleanup();

return 0;
}

```

Client code(client.c)

```

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <winsock2.h>

#define PORT 8080

#define MAX 10

int main()

{

    WSADATA wsa;

    SOCKET serverSocket, clientSocket;

    struct sockaddr_in serverAddr;

    int n, matrix[MAX][MAX];

    int i, j;

```

```
int sum = 0, trace = 0;

int upper = 1, lower = 1, diagonal = 1;

char result[1024];


WSAStartup(MAKEWORD(2, 2), &wsa);


serverSocket = socket(AF_INET, SOCK_STREAM, 0);


if (serverSocket == INVALID_SOCKET)
{
    printf("Socket Creation Failed\n");
    return 1;
}


printf("Socket Created Successfully...\n");


serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = INADDR_ANY;
serverAddr.sin_port = htons(PORT);


if (bind(serverSocket,
        (struct sockaddr *)&serverAddr,
        sizeof(serverAddr)) == SOCKET_ERROR)
{
    printf("Bind Failed\n");
    closesocket(serverSocket);
    WSACleanup();
    return 1;
}


printf("Bind Successful...\n");


listen(serverSocket, 5);


printf("Waiting for Client...\n");


clientSocket = accept(serverSocket, NULL, NULL);
```

```

if (clientSocket == INVALID_SOCKET)
{
    printf("Accept Failed\n");
    return 1;
}

printf("Client Connected...\n");

recv(clientSocket, (char *)&n, sizeof(n), 0);
recv(clientSocket, (char *)matrix, sizeof(matrix), 0);

printf("\nReceived Matrix:\n");

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        printf("%d ", matrix[i][j]);

        sum += matrix[i][j];

        if (i == j)
            trace += matrix[i][j];

        if (i != j && matrix[i][j] != 0)
            diagonal = 0;

        if (i > j && matrix[i][j] != 0)
            upper = 0;

        if (i < j && matrix[i][j] != 0)
            lower = 0;
    }
    printf("\n");
}

if (diagonal)
    strcpy(result, "Matrix Type : Diagonal Matrix\n");

```

```
else if (upper)
    strcpy(result, "Matrix Type : Upper Triangular Matrix\n");
else if (lower)
    strcpy(result, "Matrix Type : Lower Triangular Matrix\n");
else
    strcpy(result, "Matrix Type : General Matrix\n");

char temp[200];

sprintf(temp, "Sum of Elements : %d\nTrace : %d\n",
    sum, trace);

strcat(result, temp);

printf("\n%s", result);

send(clientSocket, result, strlen(result) + 1, 0);

closesocket(clientSocket);
closesocket(serverSocket);
WSACleanup();

return 0;
}
```

Output

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.26288.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\prart\OneDrive\Documents\assignment3>client5.exe
Socket Created Successfully...
Connected to Server...
Enter order of square matrix: 3
Enter matrix elements:
1 2 3
3 5 4
6 2 1

Result from Server:
Matrix Type : General Matrix
Sum of Elements : 27
Trace : 7

C:\Users\prart\OneDrive\Documents\assignment3>
```

```
C:\Users\prart\OneDrive\Documents\assignment3>
Socket Created Successfully...
Bind Successful...
Waiting for Client...
Client Connected...

Received Matrix:
1 2 3
3 5 4
6 2 1

Matrix Type : General Matrix
Sum of Elements : 27
Trace : 7

Process returned 0 (0x0)   execution time : 112.922 s
Press any key to continue.
```

Conclusion

The matrix analyzer server was successfully implemented and tested with a 3×3 upper triangular matrix.